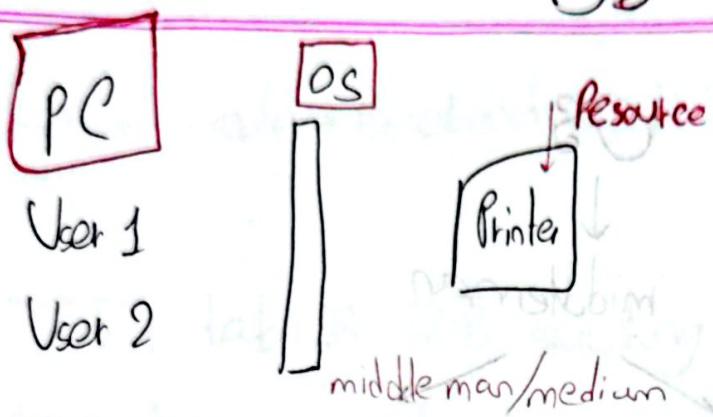
OS 15.6.26



এরকম যেন না হয় User 1 Printer use করার জন্য command দিয়েছে আর wait করছে

⇒ Printer এ যদি more than one printing command আসে from more users, then OS handle করে যে command আগে আসছে সেটাকে print করা

⇒ Priority basis এ printing টা handle করে OS

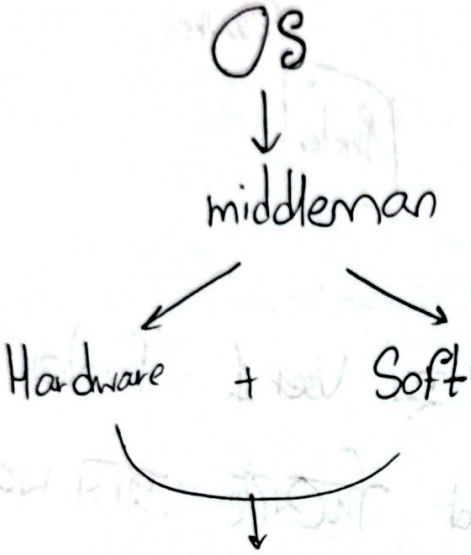
⇒ Resource management → OS

OS classroom code: 7vljfqrw

Lab: Linux command

Ref: book Modern OS

Scanned with CamScanner



⇒ OS provides smooth user experience

OS না থাকলে print button click করলে print হত না.

⇒ Resource management → OS

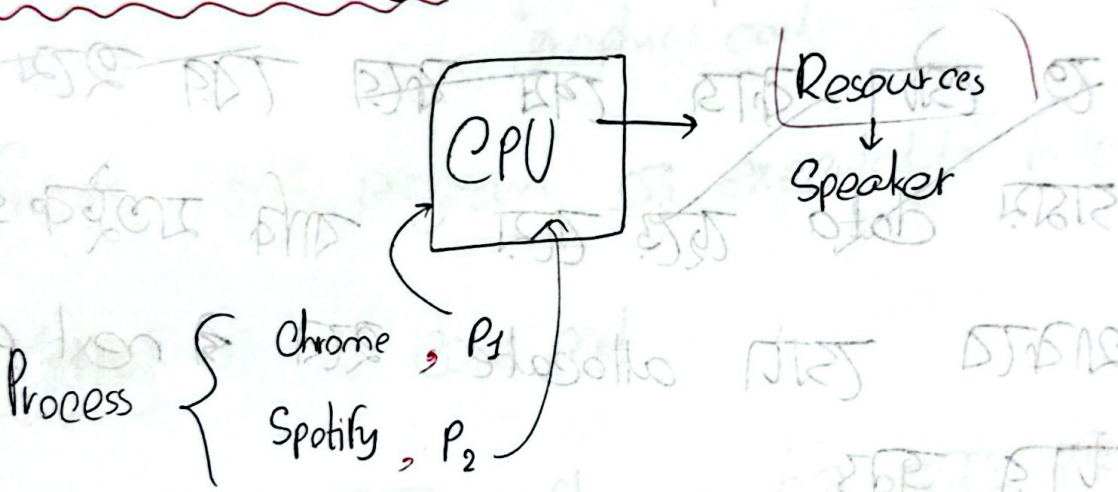
Job যেটা আগে আসবে সেটা আগে handle করবে

⇒ Memory management → OS

⇒ Kernal mode → Backend এ কীভাবে program টা কাজ করবে

আমাদের tab এর মধ্যে switching এর মাধ্যমে OS runs several process at a time

Time multiplexing :



P_1 sent at 10:00am
P_2 " " 11:00 am
to CPU

⇒ But if P_1, P_2 both sent to CPU at 10am Particular time এ দুইটা diff. process CPU তে যাচ্ছে যারা একটা particular resource use করতে চাচ্ছে, এক্ষেত্রে প্রথমে যেটা

পাঠানো হয়েছে তার জন্য Resource(Speaker) টা কাজ করবে.

Space multiplexing:

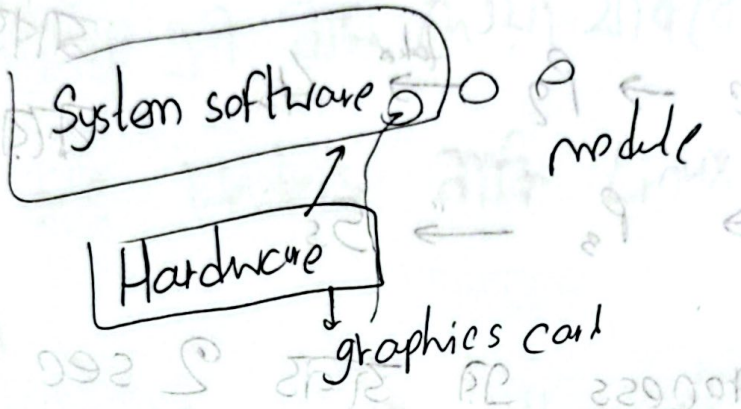
Res Particular process টা যে data নেয় memory তে সেটা কাজ শেষ করে বের হয়ে যাওয়ার সময় data ছেড়ে দেয় বাকি যতটুক space থাকবে সেটা allocate হবে ঐ next process টার জন্য

Questn
OS কী কাজে লাগে

Sandhamaam

17.6.26

⇒ System software প্রত্যেকটা Hardware এর জন্য একটা করে module create করে রাখে



Process: একটা program এর executable file

Kernal: OS এর core part

⇒ Particular কোন software এর জন্য কোন hardware কে access দিবে সেটা kernal করে

⇒ Kernal basically hardware এর resource টা maintain করে at core level

⇒ Bootstrap load হয় EPROM থেকে (secondary memory)

Scanned with CamScanner

Multiprogramming

'0' sec → P_1 takes 3s

'1' sec → P_2 takes 4s

'2' → P_3 → 5s

Time sharing এর মাধ্যমে কাজ টা করে

তিন process এর জন্য 2 sec করে নেয়

System call: user mode থেকে Kernal mode এ যাওয়া যাবে

```
User
    int a = 5
    int b = 3
```

user mode এ summation বের করার দরকার নাই তখন Kernal mode এ গিয়ে summation টা বের করে user mode এ return করবে then user mode এ print করে দিবে or Kernal mode থেকে directly user mode

এ print দিয়ে দেয় তখন return করার দরকার হয় না

Exam ⇒ System call এর নাম দেওয়া থাকবে গুলো কী কাজ করে Windows নাকি Linux এ এগুলো নামবে

⇒ What is system call with example, কীভাবে হয়

Exam ⇒ Booting এর example কীভাবে হয়

⇒ Dual mode - Kernal mode and User mode

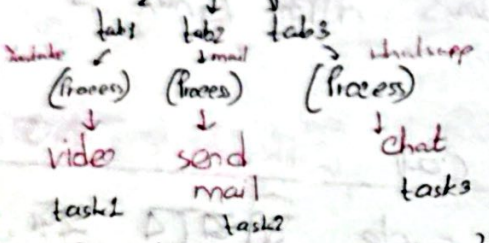
Shameha maam

22.6.26

Accessibility

Program

Chrome



⇒ 3 টা tab কে একেকটা process ধরলাম

⇒ video, send mail এসব task

⇒ Program এর under এ অনেকগুলো process থাকবে। সেই process আবার অনেকগুলো task handle করে

P_1 and P_2 দুইটা process hard drive এ Write করতে চাচ্ছে

Hard drive

P_1 → process wants to write
I want to eat rice

P_2 → I go to MIST

⇒ Hard drive এ একটা sentence ই লিখা যায়

⇒ P_1 and P_2 at a time Hard drive এ Write করতে চাচ্ছি।
same time এ দুইটা process কে একত permissn দিলে ~~override~~ override হয়ে যায়

Hard drive → I go to MIST rice (দুইটা line এ mixed for

Problem- দুইটা process কে same time access দিলে override হচ্ছে

Solutn - Accessibility

Accessibility কে 4 ভাগে ভাগ করা হয়

Access → Ring

R_0 R_1 R_2 R_3

4 টা level এর

R_0 has super power → সবচেয়ে বেশি imp

↳ Kernal mode

সব type এর imp task - error handling, data access, bootup system এসব complex কাজ R_0 তে হয়

R_1, R_2 → system driver

R_3 → সবচেয়ে কম access

বারবার permission নেওয়া লাগে

4 টা level এর মধ্যে সবথেকে imp R_0, সবথেকে less imp R_3.

R_1 and R_2 are not available in modern OS

Modern OS এ 2 টা mode R_0 and R_3

R_0 → Kernal mode

R_3 → User mode

Why 2 mode? security concern (No Random access)

R_3 এর মধ্যেই সব 3rd party concern থাকে কিছু Junk file এ চলে আসে R_3 permission নেয় R_0 থেকে entrace পাবে কি না, এই permission না থাকলে যেকোনো junk file দুকে important data delt করে দিবে OS থেকে

দুইটা level না থাকলে no access permission যার যেটা access লাগবে Kernal mode এ ঢুকে যায়, বের হয়ে আসে

কোন mode এ কতক্ষণ আছি কীভাবে বুঝব?

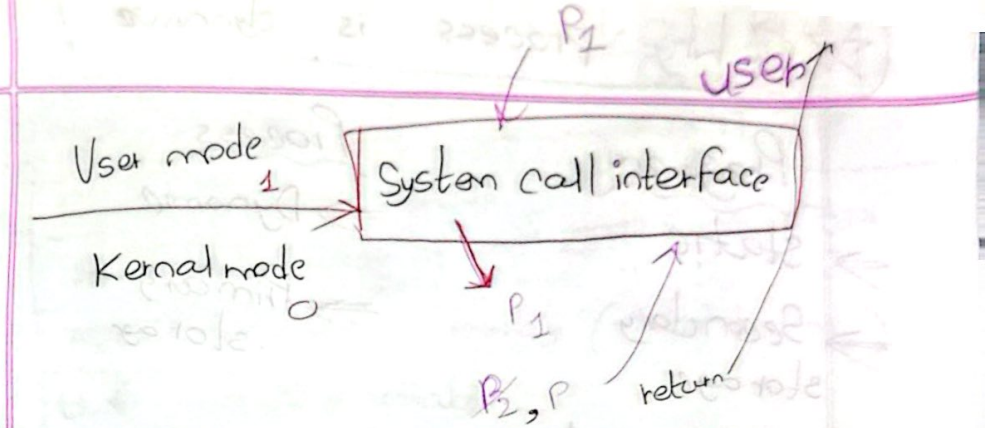
Bit দিয়ে change করা হয়

0 bit হলে Kernal mode

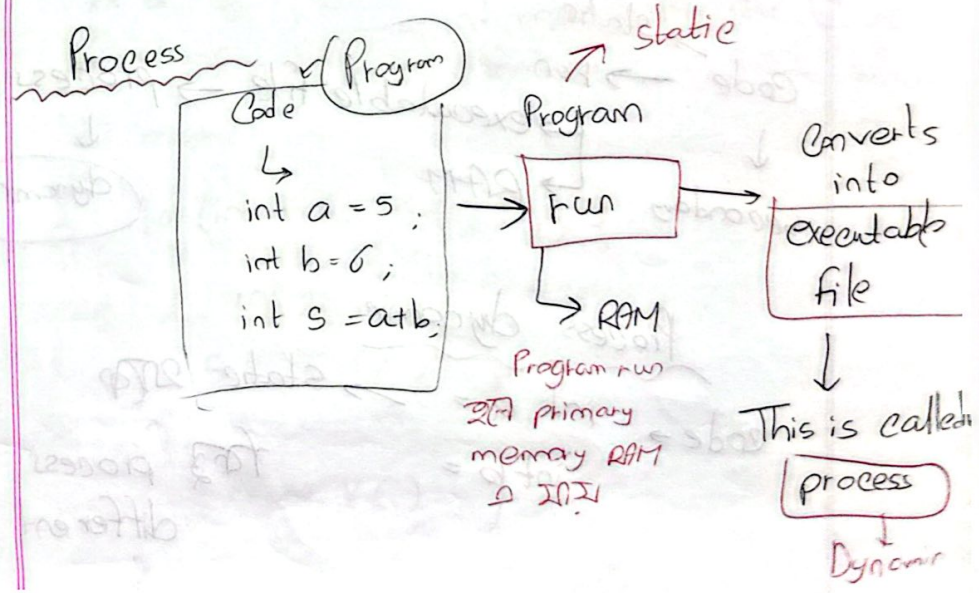
1 " " User "

একটা program run হচ্ছে, diff process এ ভাগ হলো, process গুলো task handle করবে, একটা task harddrive এ access এর আগে 0/1 bit চেক পাঠায়।

কোনো process 1 mode থাকলে সে 0 mode এ যেতে চাই, তখন flag এর মান change করে then system call interface এ যাবে (Permiss" খুঁজে) after that Kernal বুঝবে যে access দিবে



Interface is a gateway. Kernal mode এ ঢুকতে permiss" নিবে



☆☆ Why process is dynamic

Program	Process
⇒ Static	⇒ Dynamic
⇒ Secondary storage	⇒ Primary storage
⇒ Takes input from user	

Code → (static) → run → executable file → process → dynamic

↓ secondary → RAM

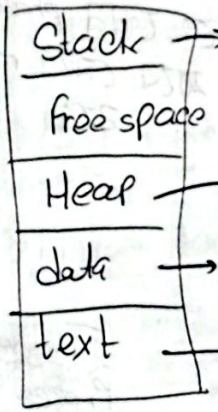
Process dynamic

code → int a = → static হয়ে থাকে

int b = → তবু process different

Process Structure

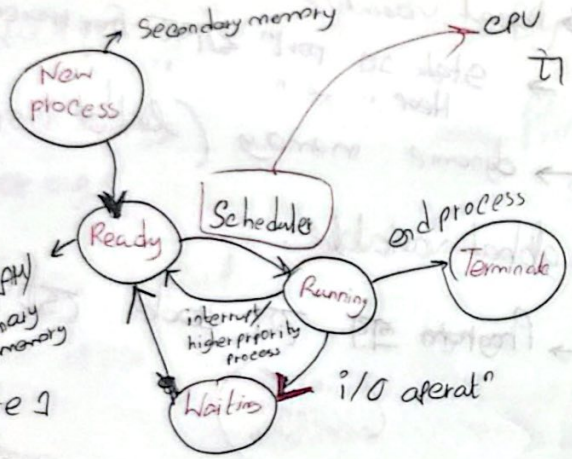
Stack overflow - free space না থাকায় হয়




```
int a = 5.
int b = 6.        → int main এর বাইরে global var.

int main()
{   for(int i=0; i<n; i++) {
        int p;
        S.O.S = ("p");
    }
    v = new V();  }
}
```

Process State Diagram



CPU তে কোন process টা যাবে সেটা manage করে

Ready state এ একের অধিক process থাকতে পারে

Running phase means process টা OS এ চলছে (CPU তে)

Waiting: input output এর কাজ করবে

⇒ Waiting থেকে Ready তে যাবে then running

Terminate Process কীভাবে করে বের হয়ে যাবে

process কি ভাবে job নেয়

Scheduler

Long term scheduler এর exist করবে না

1) Short term: Ready → Running

2) Long term: job queue → ready queue → CPU

3) Medium term:



Secondary storage

P_1
P_2
P_3
P_4

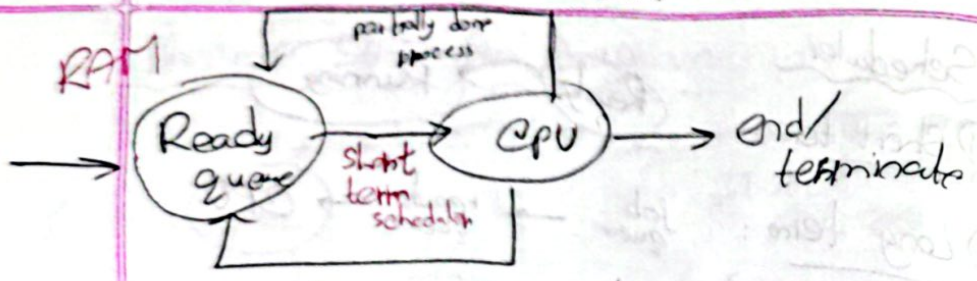
⇒ Long term scheduler যে... আগে ছিল সেটাতে sh... term scheduler exist ...

⇒ এরকার OS এ mid ter... scheduler handle ...

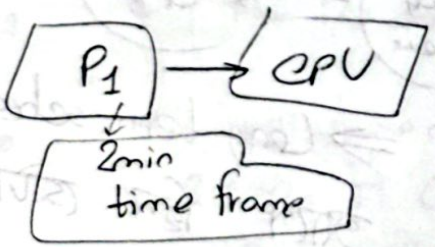
Scanned with CamScanner

medium term interrupt হলে আবার ধরে

swap out



context switch/ Quantum time



⇒ P_1 কে time দিলাম 2min CPU তে কাজ করে বের হয়ে আসতে হবে But P_1 requires 10min to complete.

This is called swap out

কোনো একটা particular process CPU তে যদি quantum time শেষ হয় CPU থেকে বের করে দিবে

বাকি process গুলো 2min শেষ করে বের হলে P_1 আবার যাবে

Swapout for 2 reasons

1) I/O operation এর জন্য back করে ready queue তে

2) কোনো একটা higher priority process যদি চলে আসে

Quest"

Accessibility থেকে Process Scheduler

Medium term

interrupt / ~~high priority~~

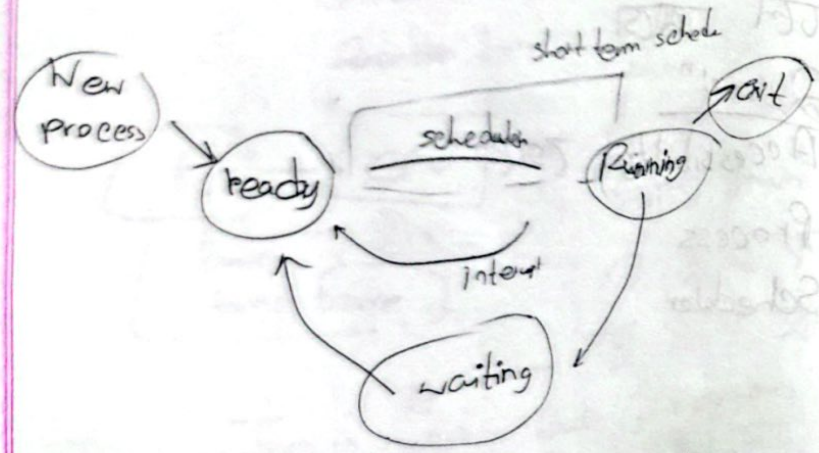
swapout

Ready queue

CPU

i/o

swapout



25.6.26

⇒ Quantum time পর পর particular process কে CPU থেকে বের করে দেয়

What is context switch?

We are

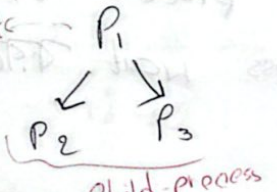
Particular process definite time পরপর CPU তে যুক্ত বের হচ্ছে → context switch

Exam ☆☆ Explain scheduler with scenerio, example, state diagram সহ

scenerio → with process 5 টা process diff. time এ, quantum time fixed then explain

Parent - child process

Parent process



Cake bake
↓
ing. collect
mixing

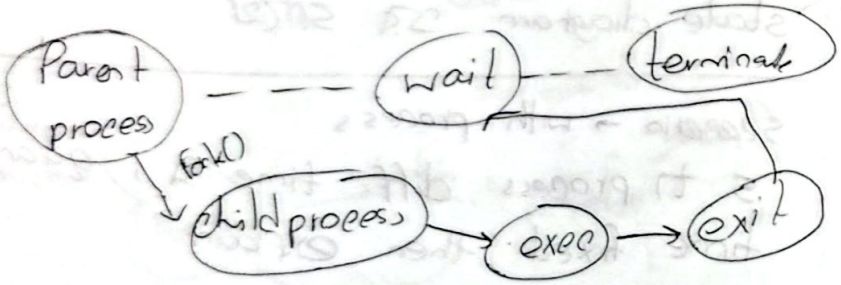
একটা parent process কে ছোট ছোট process এ ভাগ করে

Function

⇒ fork() → applicat" child

⇒ exec() → execute "

⇒ wait() →

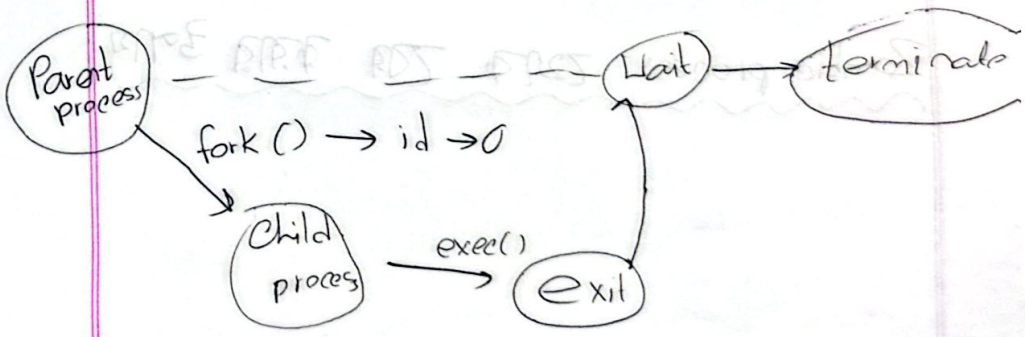


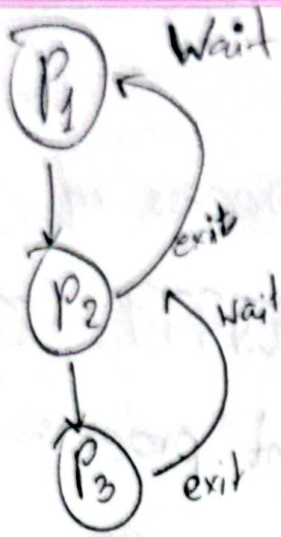
⇒ Parent process তার কাজ ভাগ করে দিয়েছে child process কে তাই এখন parent process wait করতে করতে exit এ যাবে।

⇒ Child process execute হয়ে exit হয়ে parent process কে wait এ থাকা তাকে report করবে। Then terminate হয়ে যাবে

⇒ Every process has unique process id একটা distinguisher থাকা দরকার যে বুঝাবে যে এটা child নাকি parent process. Parent process এর process id 0 থেকে বড় Child " " " " 0 " ছোট

⇒ fork() থেকে id → 0 return করবে to ensure child process is created (এটা





What is zombie situation / process

Childprocess execute ৩ হতে পারছে না, parent কে report ও করতে পারছে না, তখন parent process wait করতেই থাকে

Zombie process থেকে বের করার উপায়:

Sameha maam

24. 6.26

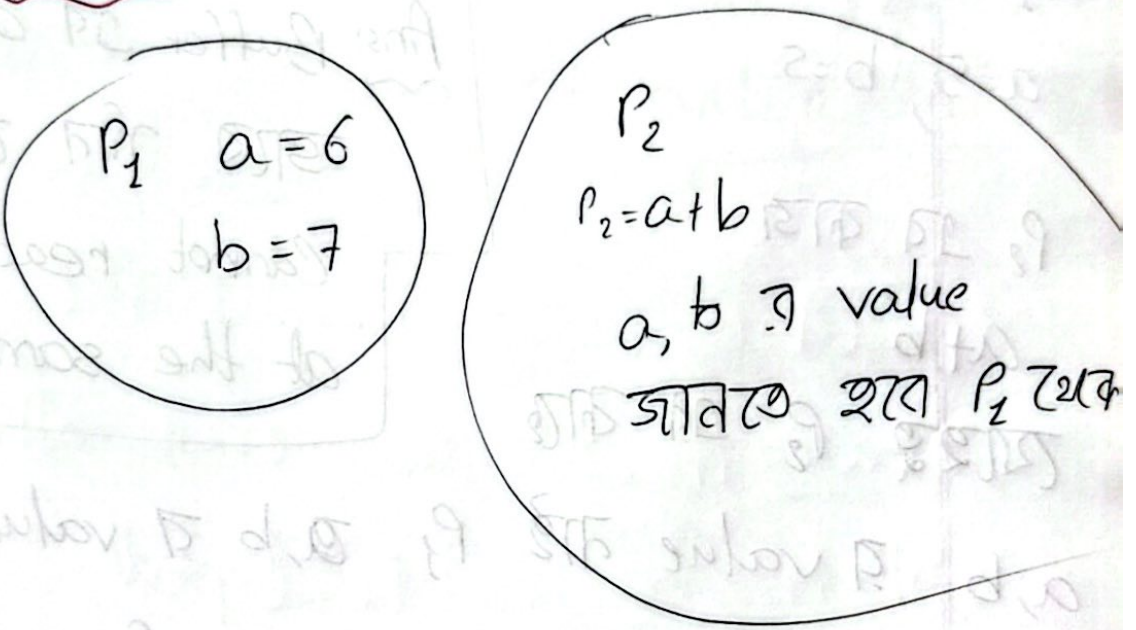
Inter process communicat"

1) Independent → কাজ করার জন্য অন্য কোনো process এর সাথে communicat" লাগেনা

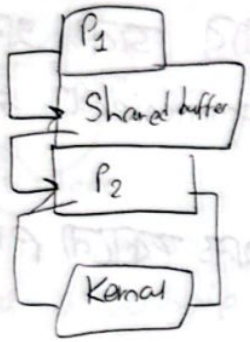
$P_1 \rightarrow a+b$

a+b করার জন্য অন্য কোনো process থেকে info নেওয়া লাগছে না

2) Co-operative



Shared memory



→ hard drive এ দুইটা process একসাথে হলে বাজেনা হয় but buffer টা কে share করা যায় P_1, P_2 তে. How?

Ans: Buffer এর access এমন ভাবে দিব যেন both cannot read and write at the same time

P_1 এর কাছে
a=5, b=5

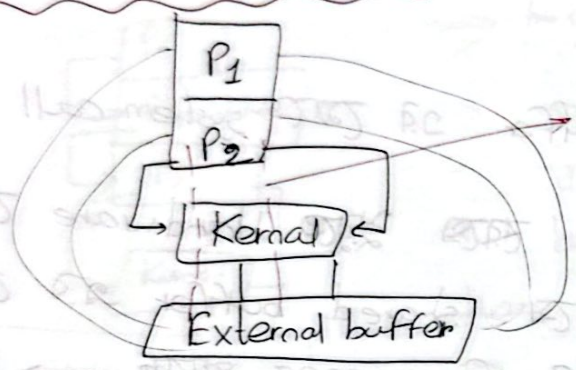
P_2 এর কাজ
a+b
যেহেতু P_2 এর কাছে a, b র value নাই P_1 a, b র value shared buffer এ রাখবে তারপর P_2 নিবে

⇒ Shared buffer দিয়ে bg data exchange করা হয়

⇒ shared buffer এর জন্যে system call হবে.

P_1 value রাখার জন্যে Hardware কে জানায় যে shared buffer এর কোন portion P_1, P_2 access পাবে → এটার জন্য মুহূর্তে একটা system call হয়

Message passing



→ External buffer ত use করতে হলে every time Kernal এর thru তে যেতে হবে। যেহেতু Kernal দিয়ে যাবে every time system call লাগবে, as it is an external path

No issue of override cuz Kernal এর thru তে যায়, buffer এ access

কিভাবে এগুলো প্রত্যেকবার ask করে

⇒ No big data utilize

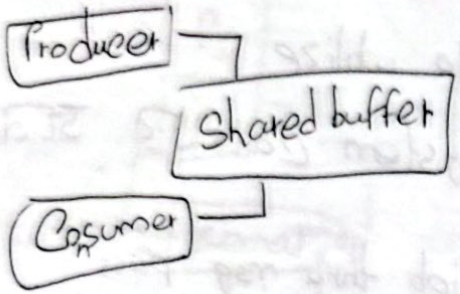
প্রত্যেকবার system call এর জন্য lag হয়

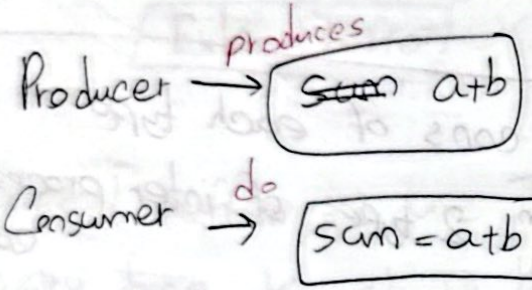
⇒ do certain job thru msg pass

→ pros and cons of each type

Exam ☆☆ → diff betn 2 types of inter process communicat

OS problem → producer/consumer problem

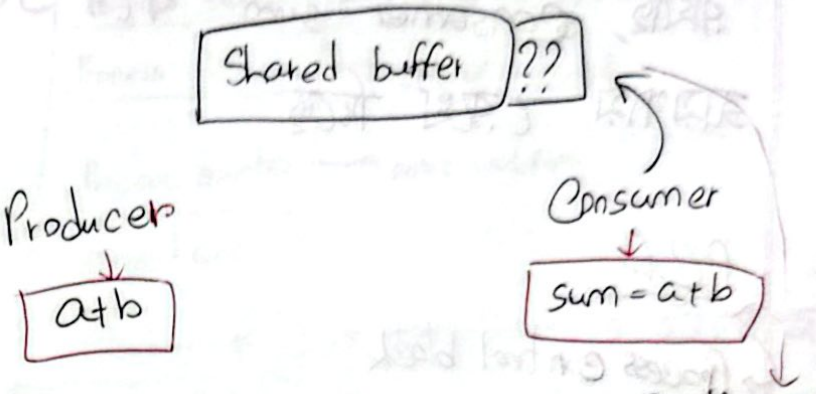




Producer a+b কে shared buffer
এ রাখে

Shared buffer is of fixed size

Consumer সেই জায়গা a, b র value buffer
আছে, সেখানের থেকে access করে



⇒ Consumer repeatedly searches if there are any more data in the shared buffer

Producer - consumer এর rate না মিললে
3 types এর problem হয়

1) Overflow → producer বেশি হয়ে গেছে shared buffer এ জায়গা শেষ

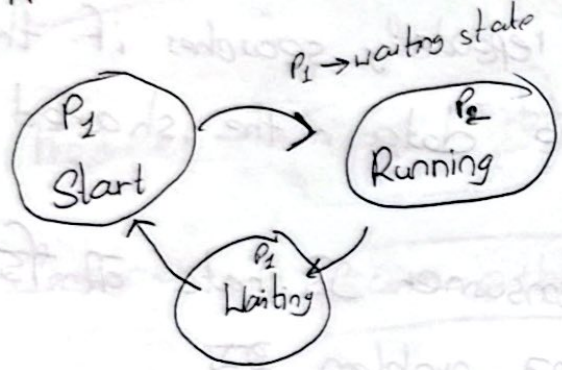
2) Underflow → producer ঠিকমতো বানাতে পারেনাই but there is ample space in buffer

3) Override → a এবং b এর value যেখানে

করছি, consumer sum করে একই জায়গায় রেখে দিচ্ছে

PCB

Process control block



Process state

Process id

Process state	→ ready / wait / running
Process id	→ unique identity
Program counter	→ next workflow
Registers	
Memory limits	→ stack/heap এ কতটুক জায়গা
List of files	

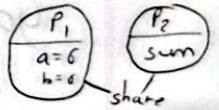
fig: Program control block

⇒ Program এর control unit এটা দিয়ে determine হয়

IPC কখন লাগে

Why we need

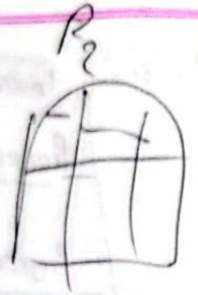
1) Informat^n sharing
2) Computat^n speed
3) Modularity → দুইটা process এর মধ্যে communicat^n



Scanned with CamScanner

Process structure

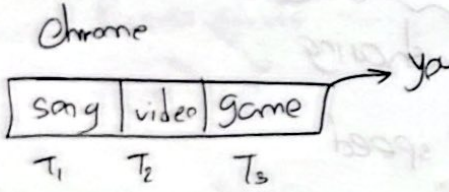
P_1		
Code	Data	file
Register	Register	Register
Prog. Counter	Prog. counter	Prog. counter
stack	stack	Stack
S	S	S



Thread

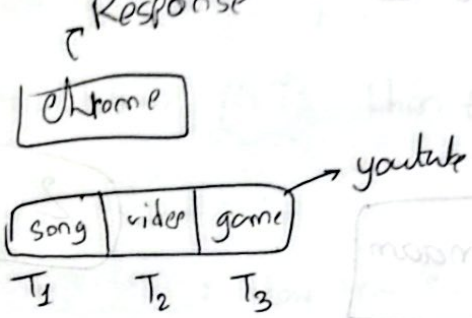
দুইটা বা তিনটা process একসাথে same ভাবে জায়গা মিলে করবে (process structure)

⇒ প্রত্যেকটা process কে thread এ ভাগ করতে চাচ্ছি



youtube process থেকে thread নিয়ে তিনটা diff. কাজ করাচ্ছি,

এটার কারণে data lag কমে যায়

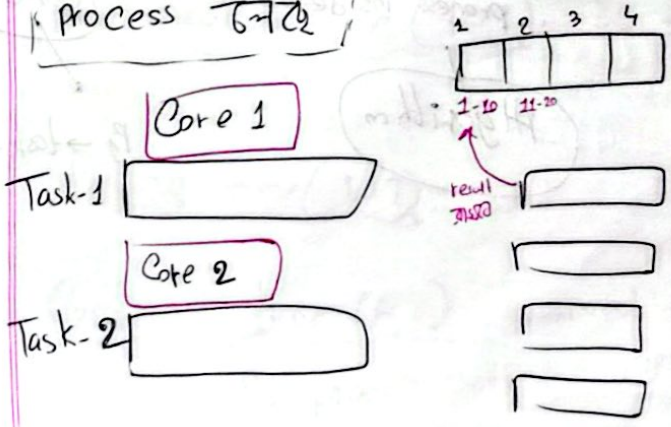


1) Response 2) Resource 3) Economy 4) Scalibility

একইসাথে অনেক গুলো কাজ accomplish thread create করে

5) Concurrency
at a time diff. process চলছে

6) Parallelism
Task → দুইভাবে
Task and Data



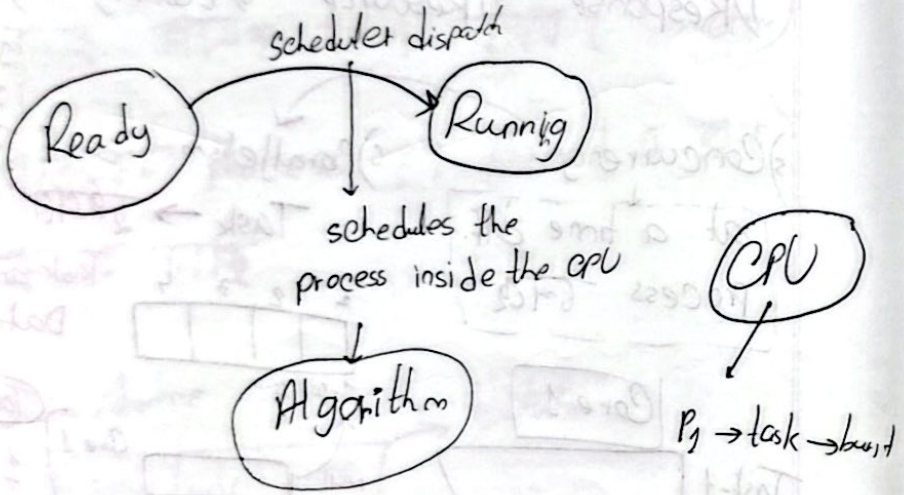
Parallelism is under Concurrency

What is parallelism?

multiple core ↴ Task divide করে দিবে

Sameha maam

2. 7.26



Burst time: কতটা task CPU তে execute হয়ে বের হবে

Arrival time (AT): When the process arrives in system

Burst time: How much time process spends in CPU

Completion time (CT): When a process is done with execution

Turn around Time (TAT): The time from arrival to completion

Waiting time (WT): WT = TAT - BT

Response time (RT): arrival time of process when the process gets the CPU for the very first time

Scanned with CamScanner

Scheduling length (SL): Total duration of time (max (CT) − min (AT))

max (CT) → among all process

min (AT) → among all places

Throughput: measurement of operations per unit of time

Preemptive: Example

P_1 ready queue তে আসছে, P_1 arrived in memory 0sec এ, So P_1 is in ready queue. P_1 এই কাজ summation pint and it takes 3sec to do it's job. This 3sec is burst time. কাজ করতে যতটুকু time লাগে → 3sec = Burst time.

Now P_1 goes to running then terminate after 3sec of burst time → This is complet" time

Let P_1 is in running. Then a higher priority process than P_1 (P_2) arrives. P_2 → (5) sec. Then P_1 moved to waiting the ready.

P_1 arrived at 0sec and in running 2sec then for P_2 it went to waiting then ready at 2sec then in running to complete it's remaining 1 sec. So total time = 5sec

→ This is turn around time

Scanned with CamScanner

P_1 → arrive → 0sec
P_2 → " → 0sec
} দুইটার priority same

P_1 আগে আসছে, P_1 কে Running এ থাকবে for 2sec

P_1 → burst time = 3sec
P_2 → " = 5sec

P_1 কে Running এ থেকে 3sec কাজ করে terminate
P_2 CPU কে পাবে 3sec পরে
→ This is Response time

Preemptive: forcefully কোনো process কে CPU থেকে Waiting এ নিয়ে আসতে (Running) পারি

Ex: যখন কোনো higher priority process আসে

Non-preemptive: বের করে আনতে পারবেনা কোনো process কে Running এ

Ex: একবার CPU পেলে যে পুরাটা শেষ করে then terminate. কখনো Running state থেকে বের হয়না

Read book

Modern OS slideshow
Operating System concepts
Chp-2,3

Scanned with CamScanner

CT syllabus → upto Monday's class
upto Parallelism

Lecture + slide + book

Process কী
Child "
zombie "
PCB
Inter process Communicat"
Parallelism

scenerio পড়ে
বুঝতে হবে কোন
topic then
Write theory of
that topic, concept
Relate scenerio. that the
this process

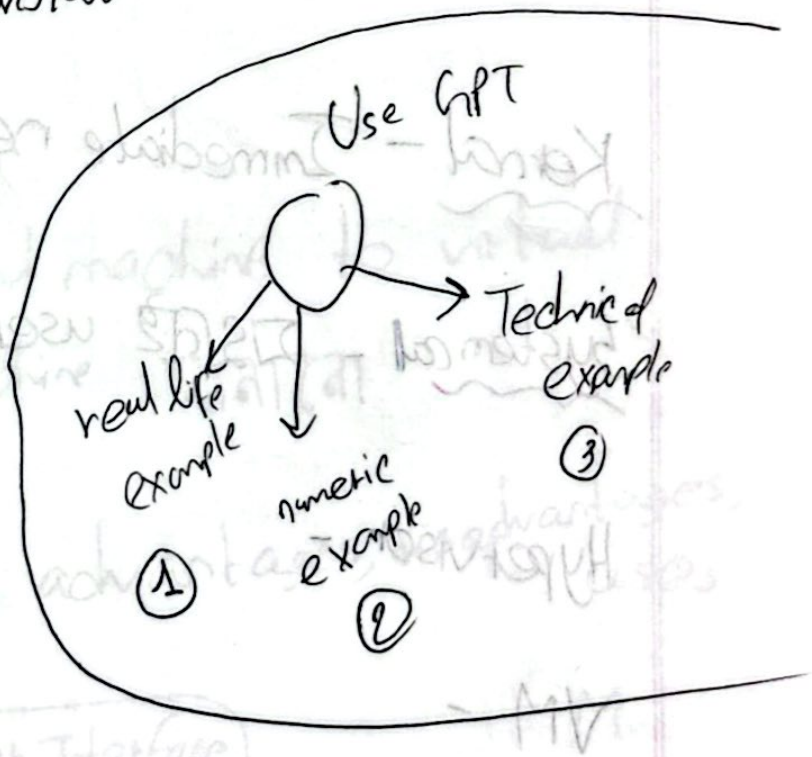
Khaled Sir & 4th week 6.7.26

Part-B

Highlighted topics of Part-B

Self study the Keywords

- Virtualization
- Virtual Machine
- Memory Management
- Disc scheduling
- Cloud and cloud revolution
- Deadlock
- Bottleneck



Process → CPU
→ I/O

Exam

Justify how CPU utilization is one of the main function of OS

CPU থেকে যখন process কে I/O তে দিল, CPU idle এ থাকে. CPU কে idle বসে যাবে না, এমন ভাবে OS কে design করতে হবে যেন CPU তে একটা পর একটা কাজ লেগেই থাকে

Kernal - Immediate next layer of hardware

Systemcall - যখনই user কোনো একটা process

Hypervisor -

Memory management

VM -

Dual boot

Virtualization

Cloud

4th week | Khaled Sir | 9.7.27

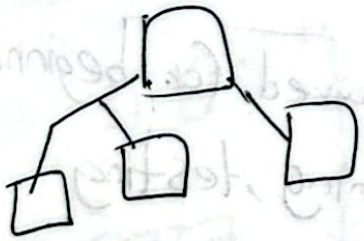
Khaled Sir 2 weeks

CSE-207

↳ Virtualizat^n
VM
Cloud

↳ Memory management
initials
Two methods

Virtualizatth → Physical machine to virtual machine বানানো

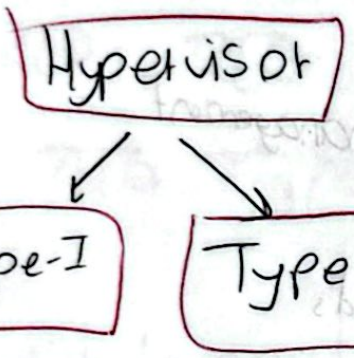


advantages, disadvantages, uses



→ manager

Hypervisor: Physical machine থেকে VM গুলোকে ঠিকমতো কাজ করায়, দুটা process, টা maintain করে



⇒ native/pure

⇒ No OS required

Type-I hyp. will work as OS and also manages VM

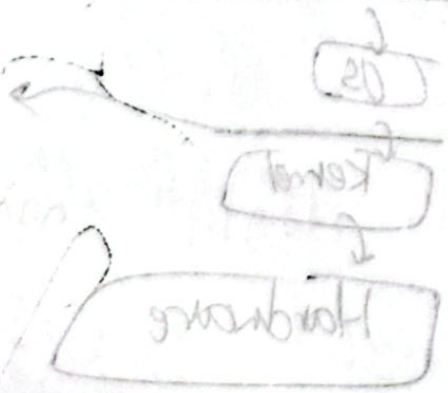
⇒ Mostly used

⇒ An OS already present

OS দ্বারা influenced হয়।

⇒ Not native

⇒ Required for beginner learning, testing



Scanned with CamScanner

Cloud 3 types

Cloud → Iaas
→ Paas
→ Saas

NIST 5 essential characteristics

Exam

Market ১ 10 টা New OS launch হয়েছে Manager wants a report কোনটা কিভাবে perform করছে। How you can solve this problem

Faster

Ans: OS can be run as

1) OS
এটা করলে 10 টা OS independently set করতে হবে (Time)^n লাগবে

2) Dual boot
দুটোর্ড্যাক করা time লাগবে (Time)^n

3) VMware
Software টা থাকে PC র .iso file থাকে 10 টা .iso file থাকে setup and tests

Part-B ।। Compulsory

Memory management in OS

Memory hierarchy

- Access time
- Cost
- Storage

⇒ যত CPU র কাছের memory তত fast access time তত বেশি cost.

Exam Ques: Memory কিনবে, কোনটা কিনবে, Depends on the user's need Criteria দিয়ে দিবে, Budget depend করতে বলতে পারে

Multiprogramming

RAM এ multiple process একসাথে রাখা(না)

Degree of multiprogramming

Degree বাড়াইতে RAM এর size increase করতে হবে

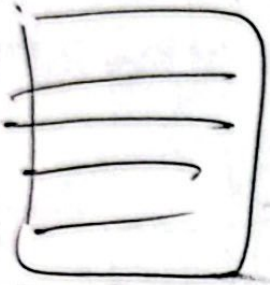
Exam ☆☆

Degree | RAM size

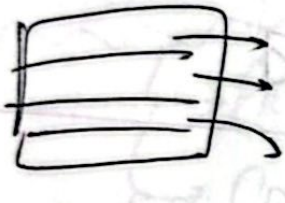
Are these 2 proportional or not

Ans: Yes. RAM size যত বেশি তত বেশি process একসাথে run করানো যাবে RAM

Contiguous allocation



Memory segment করা



sequential way তে process গুলো
কে মেমরি যেতে হবে

Contiguous

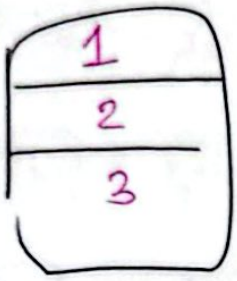
Partioning এর basis
এ 2 types

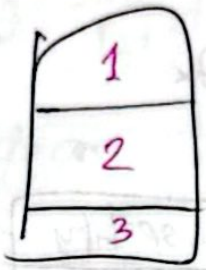
Fixed/Static

Dynamic/Variable

No. of partition টা fixed
কিন্তু Partition এর কতটুকু size
সেটার not mentioned

Scanned with CamScanner





fixed partitioning বলে দেওয়া 3 টা partition. প্রত্যেকটার size ইচ্ছামতো

Dynamic partitioning

memory request না আসলে partition create করবেনা

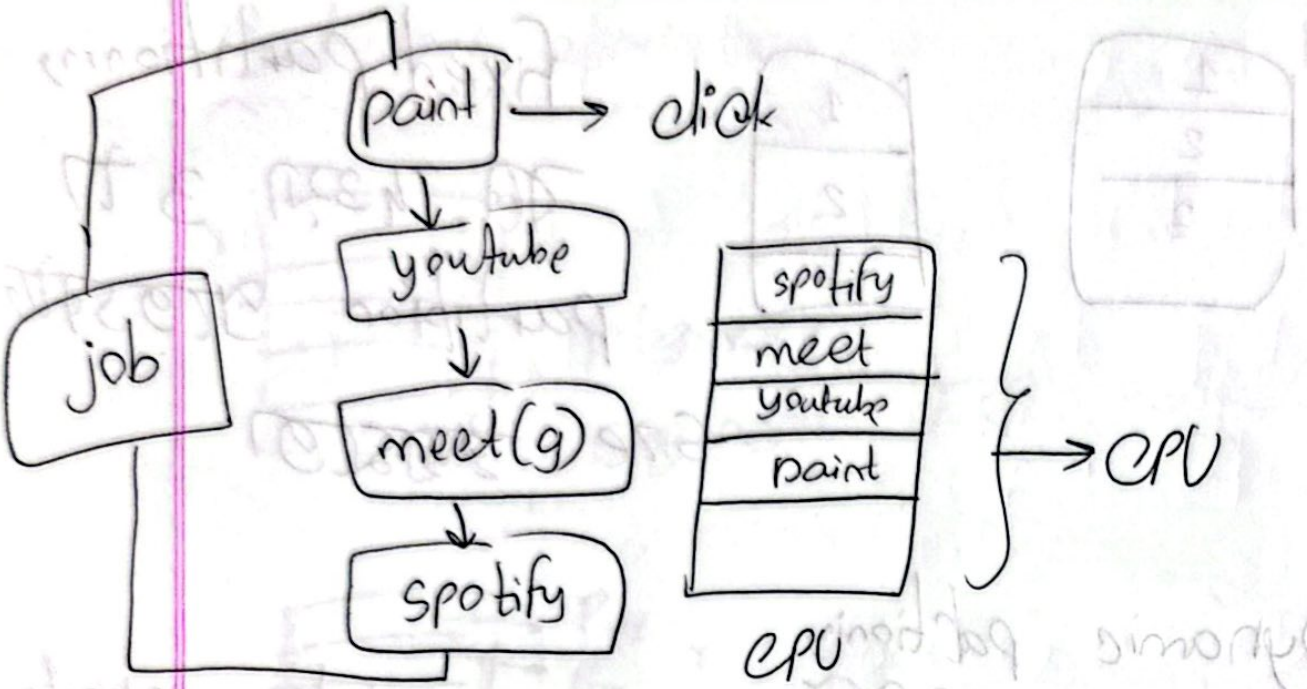
Exam Dynamic partitioning relating to Thin provisioning not Thik provisioning

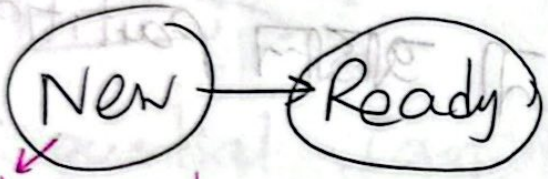
VMWare এ কত GB নিবে সেটা নিয়ে

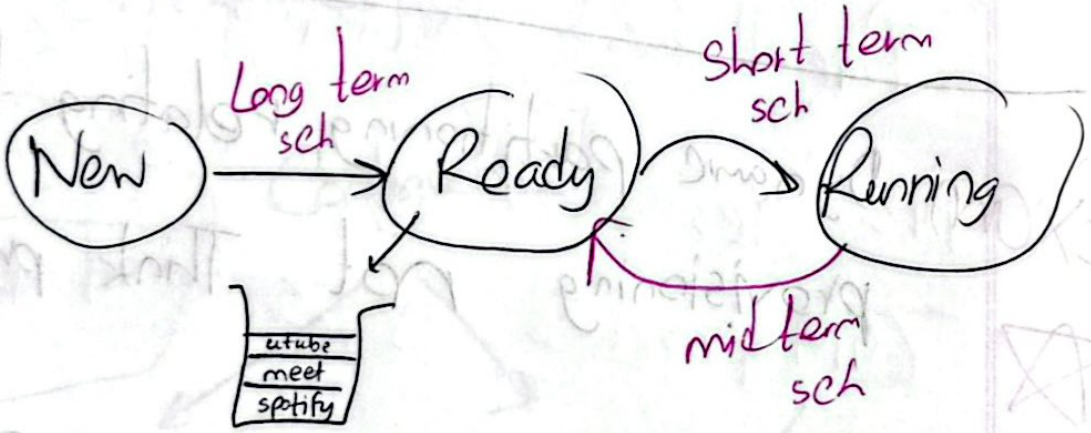
Samiha maan

Scheduling alg

20.7.26







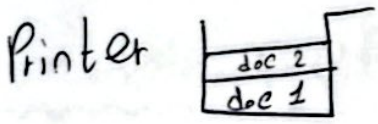
CPU তে যারা থাকে

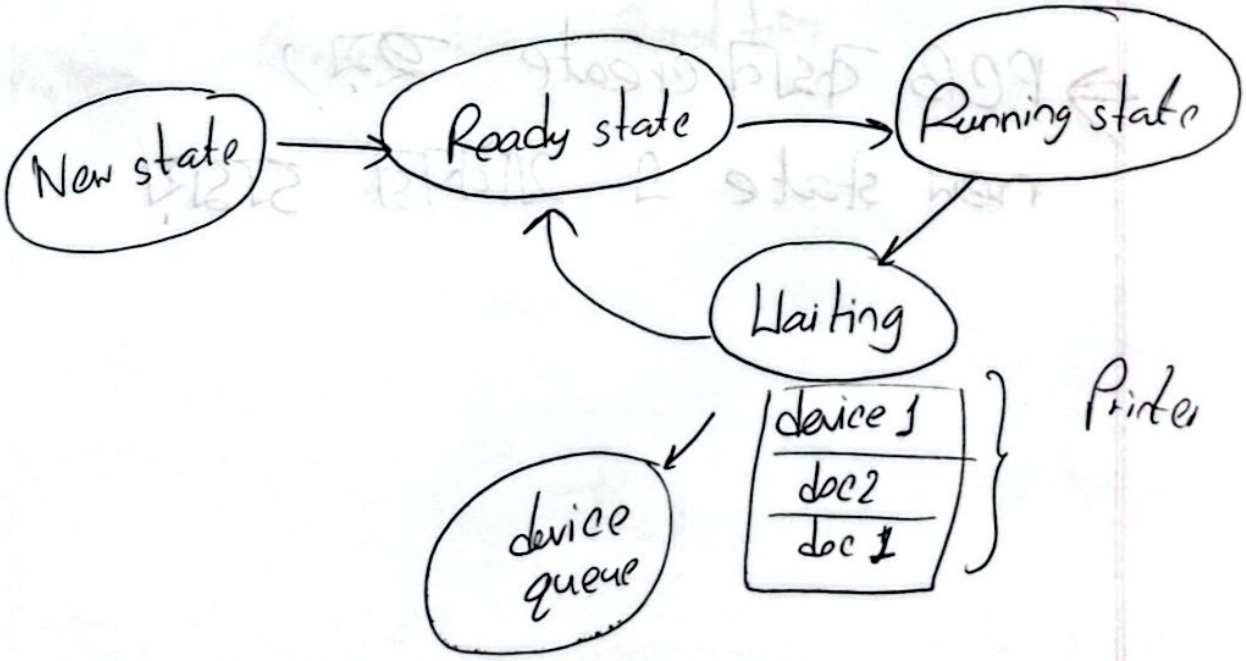
CPU তে আর জায়গা নাই এখন যদি নতুন program CPU তে

load করতে চায় তাহলে অনেক আগে যে program load হয়ে idle বসে ছিল তাকে বের করে দিয়ে নতুন program load করবে → এই কাজ টা করবে mid term seh

Every device has a particular queue

Printer





⇒ swap in/swap out কে Roling ও বলা যায়

⇒ Paint program অনেকক্ষণ ধরে idle ছিল তাই paint কে swap out করে game new program কে swap in করে

⇒ Swap space → secondary memory যেখানে swap out হওয়া process গুলো store হয়

⇒ PCB কখন create হয়?
new state এ থাকার সময়

Ready → Running

Ready to running এ কোন process যাবে সেটা handle করে কিছু algorithm

Non-preemtive → Ready to running এ গিয়ে execute হয়ে then terminate হয়ে যাবে

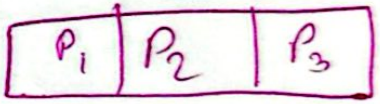
Example: FCFS

Process ~~Arrival~~ time ~~Burst~~ time

CPU তে কাজ শেষ করতে কতক্ষণ লাগে

Process	Arrival time	Burst time	Completion Time	CT - AT (Turn around time)	TAT - BT (Waiting time)
P_1	0	30	30	30 (30-0 = 30)	0 (30-30 = 0)
P_2	0	5	35	35 (35-0 = 35)	30 (35-5 = 30)
P_3	0	5	40	40 (40-0 = 40)	35 (40-5 = 35)

Avg turn around time $= \frac{30+35+40}{3} = 35$



Gantt Chart:

Avg waiting time $= \frac{0+30+35}{3} = 21.66$

Scheduling length $= \overbrace{\max(CT) - \min(AT)}^{\text{CPU কতক্ষণ রেখি}} = 40 - 0 = 40$

Throughput $= \frac{\text{no. of process}}{\text{Sch. length}} = \frac{3}{40}$

Scanned with CamScanner

Non preemptive

⇒ Depends on process arrival

⇒ if same AT then go for process id

FCFS → first come first serve

⇒ Non preemptive sch. alg.

→ P_1 আগে আসলে তাকে serve

⇒ P_2 আসলে P_1 কে terminate করে then P_2 কে নিবে

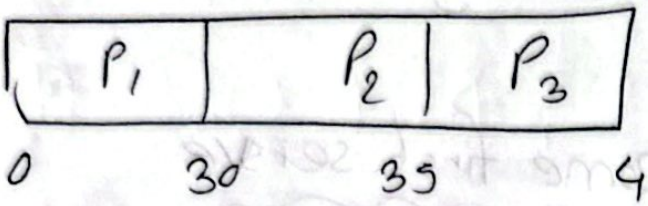
If P_1, P_2, P_3 comes at the same time তখন process id এর উপর depend করে কাকে আগে serve করবে

Exam

Quest এ scenario দিয়ে AT BT দেওয়া, তেমন তেমন table বানাতে হবে। Then prepare gantt chart Then find avg waiting time

Gantt Chart: Time frame

⇒ zero থেকে timer শুরু



⇒ P_2 এর burst time 5

$30+5=35$

⇒ P_3 এর burst time 5

$35+5=40$

Complete time নির্ণয় Gantt chart থেকে

⇒ Why waiting time in non preemtive

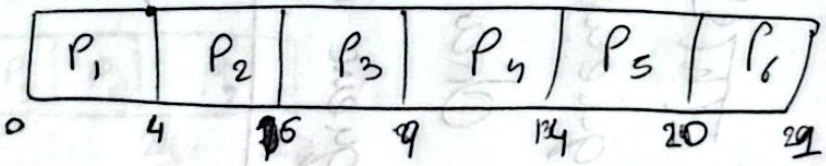
Ans: যেকোন process একবার আসলে, Ready queue তে 2 টা process halt এ, তাহলে তার waiting

FCFS Example-2

Process	Arrival time	Burst time	Completion time	Turn around time	Waiting time
P_1	0	4	4	4	0
P_2	1	2	6	5	3
P_3	2	3	9	7	4
P_4	3	5	14	11	6
P_5	4	6	20	16	10
P_6	5	1	21	16	15

Process #

Gantt chart



Avg waiting time $= \frac{0+3+4+6+10+15}{6} = \frac{38}{6} \approx 6.33$

Sch. length $= \max(CT) - \min(AT) = 21 - 0 = 21$

Avg TAT $= \frac{4+5+7+11+16+16}{6}$

$= 59/6 = 9.81$

Throughput $= \frac{\text{no. of process}}{\text{sch. length}}$

$= 6/21$

Exam ★★

Exam এ scenario based customer arrival in a restaurant. তাদের arrival time দিবে। Then avg waiting time বের করতে বলবে

How to ans

Ques দিলে Gantt chart বানাবে then Job queue বানাবে। Job queue দিলে Gantt chart fill up করবে

Job queue is based on arrival time

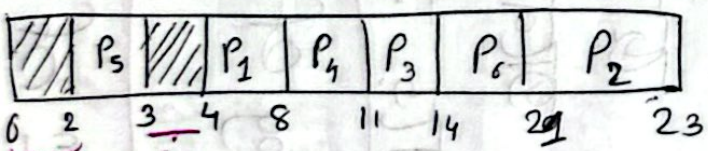
FCFS Example-3

always maintain Job queue ↓

Process	Arrival time	Burst time	CT	TAT	WT
P_1	4	4	8	4	0
P_2	8	2	23	15	13
P_3	6	3	14	8	5
P_4	5	3	11	6	3
P_5	2	1	3	1	0
P_6	7	7	21	14	~~7~~

8 sec এ current Job queue

P_4
P_3
P_6
P_2



0 থেকে 2 তে কোনো process আসেনি

Avg TAT $= \frac{48}{6} = 8$

Sch. length =

Avg WT $= \frac{28}{6} = 4.6$

যদি P_5 এর burst time 10 হত:

P_5 arrives first

	AT	BT
P_5	2	10
P_1	2	4

1st P_5 10 -- বাকি process -- গুলো 3 waiting

2nd P_1 4

যে process আগে আসছে তার burst time অনেক বড় হওয়ার কারণে বাকি ছোট process গুলো waiting time অনেক বেড়ে গেছে, তারা timelock এ পড়ে যাচ্ছে, This is called

Convoy effect

Solut of convoy effect is

SJF → shortest Job first

CPU যেন বেশি time না নেয় for a process

SJF

যেটার burst time ছোট সেটাকে আগে দিবে

এটা তখন হবে যখন অনেকগুলো process এর arrival time same. যদি diff. arrival time হয় তখন first এ যেটা আসছে সেটার BT অনেক বেশি হলেও আগে চলে যাবে as অন্যগুলো এখন আসেনি

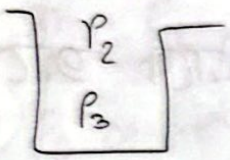
⇒ Job queue তে যদি ৫ ছোট burst time থাকে তাকে পাঠাব

⇒ Job queue যেসব process রেডি তারা already ready queue তে আছে

P_1	P_2	P_3
AT = 0	AT = 2	AT = 3
BT = 10	BT = 4s	BT = 5s

যখন P_1 আগে আসবে তাকে execute করে ফেলা হবে

P_1 চলাকালীন P_2, P_3 job queue তে।



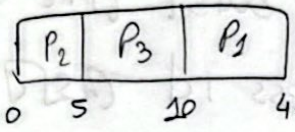
তাহলে এখন Job queue তে যেটার BT কম তাকে আগে পাঠাব, এখন arrival time matter করবেনা cuz দুইটায় ~~ko~~ job queue তে আছে

Exam ☆

Convoy effect solve করব

Example next pg →

SJF Example



Ready Queue

at time 0	P_1 P_2 P_3
at time 5	~~P_2, P_3~~ P_1, P_3

Process	AT	BT	CT	TAT	WT
P_1	0	30	40	40	10
P_2	0	5	5	5	0
P_3	0	5	10	10	5

Rules :

1) Ready queue তে যারা আছে তাদের মধ্যে shortest burst time হিসাব করতে হবে

2) যদি burst time same হয় তখন FCFS দিয়ে choose করব depending on ~~burts~~ arrival time

3) যদি arrival time আর burst time ও same হয় তখন process id যেটা

⇒ P_1, P_2, P_3 এ arrival time same আছে। সুতরাং BT যেটা কম সেটা যাবে

⇒ P_2, P_3 দুইটার BT same এখন যেটার process id

⇒ P_2 এর process id আগে এটা যাবে

⇒ এখন queue তে বাকি থাকে P_3, P_1

P_3 আগে যাবে cuz P_3 এর burst time কম than P_1

Suhena maam

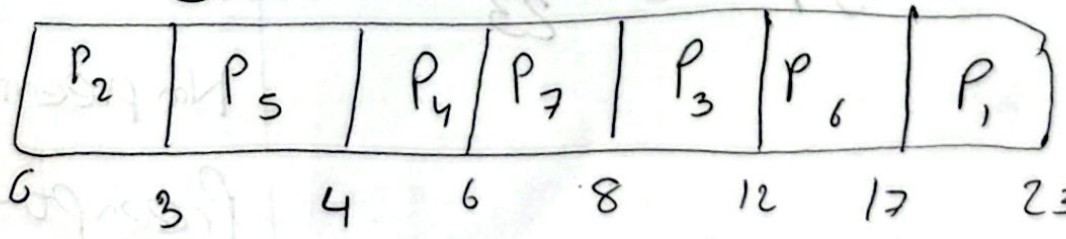
23.7.26

Non-preemptive

SJF:

Process	Arrival time	Burst time	Completion time	Turn around time	Waiting time
P_1	0	6	23	23	17
P_2	0	3	3	3	0
P_3	1	4	12	11	7
P_4	2	2	6	4	2
P_5	3	1	4	1	0
P_6	4	5	17	13	8
P_7	6	2	8	2	0

Grantt Chart



Ready Queue

at time 0 : P_1 P_2

" " 3 : P_1 P_3 P_4 P_5

" " 4 : P_1 P_3 P_4 P_6

" " 6 : P_1 P_3 P_6 P_7

avg TAT $= \frac{57}{7}$

avg WT $= 34/7$

SL $= 23 - 0 = 23$

Throughput $= \frac{7}{23}$

Issue of SJF

Burst time which is bigger waiting time is also bigger. So solve this SRTF

Non preemptive ✓✓✓

Preemptive ✓ shift

Process	AT	BT	CT	TAT	WT
P_1	0	4	4	4	0
P_2	4	3	7	3	0
P_3	6	1	8	2	1
P_4	3	5	15	12	7
P_5	8	2	10	2	0

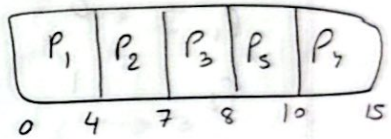
at time 0 : P_1

" " 4 : P_2 P_4

" " 7 : P_7, P_3

" " 8 : P_4, P_5

" " 10 : P_4



Avg TAT $= \frac{4+3+2+12+2}{5}$

$= \frac{23}{5}$

Avg WT $= \frac{8}{5}$

SL $= 15 - 0 = 15$

Throughput $= \frac{5}{15} - \frac{1}{3}$

Scanned with CamScanner

RT = Response time
CPU যখন first time process টা পেয়েছে — process টার arrival time

Pre-emptive

SRTF

Shortest remaining time first

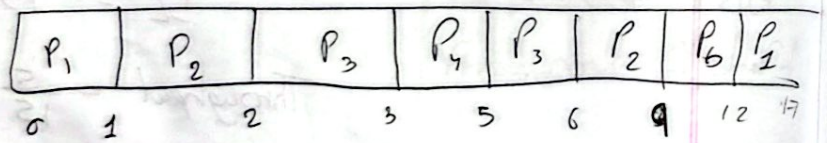
SRTF → Preemptive

Also depends on burst time

CT-AT → TAT
TAT - BT → WT

Process	AT	BT	CT	TAT	WT	RT
P_1	0	6	17	17 (17-0)	11 (17-6)	0 (0-0)
P_2	1	4	9	8 (9-1)	4 (8-4)	0 (1-1)
P_3	2	1	3	1 (3-2)	0 (1-1)	0 (2-2)
P_4	3	2	5	2 (5-3)	0 (2-2)	0 (3-3)
P_5	4	1	6	2 (6-4)	1 (2-1)	1 (5-4)
P_6	5	3	12	7 (12-5)	4 (7-3)	4 (9-5)

Gantt Chart



Ready Queue

at time 0: P_1 — $P_1 \rightarrow 6\ 5$

" " 1: P_1^5, P_2^4 — $P_2 \rightarrow 4\ 3$

" " 2: P_1^5, P_2^3, P_3^1 — $P_3 \rightarrow 1$

" " 3: P_1^5, P_2^3, P_4^2 — $P_4 \rightarrow 2\ 1$

" " 4: $P_1^5, P_2^3, P_4^2, P_5^1$ — $P_5 \rightarrow 1$

" " 5: $P_1^5, P_2^3, P_6^3, P_5^1$ — $P_6 \rightarrow 3$

যদি SJF হত তাহলে

~~ছোট~~ যেটা আগে arrive করছে তাকে আগে নিব, P_1 0 time 1 arrived so Gantt chart first 1 P_1 বসাবো আর burst time 6 তাই ততক্ষন পর্যন্ত চলতেই থাকবে। এই time এর

এখন P_2 to P_6 তারাই চলে আসবে. তাদের মধ্যে P_3 এর burst time ছোট দুইটার burst time আর arrival time same হলে যেটা process id আগে আসছে which is P_3 (FCFS) then P_5. Then বাকিগুলোর মধ্যেই ছোট burst time. P_4 then P_6 then P_2

SRTF যখন হয়

⇒ P_1 যখন 0 time এ P_1 arrive করেছে, Arrival time wise process কে নিবার্চন ready queue তে, P_1 এর burst time 6. P_1 কে 1 momentum পর্যন্ত কাজ করবে, তার P_2 আসছে at 1 momentum. তখন check করব 1 momentum এ কয়টা process আছে. তারপর ready queue বানাব,

at time 1 → P_1 P_5

⇒ Remaining time chart এ P_1 এর time থেকে remaining টা লিখব

P_1 → 6 5

P_2 এর burst time 4 and P_2 এর remaining time 5 so এখন P_2 যাবে, P_2 একবার 2 momentum পর্যন্ত। Ca2

P_3 arrived at 2 momentum

at time 2 → P_1 P_2 P_3

Remaining time chart এ P_2 update

$P_2 \rightarrow 4\ 3$

এখন কোনো P_3 এর burst time which is 1. P_1 এবং P_2 এর remaining time বেশি So এখন P_3 যাবে 3 momentum পর্যন্ত Ca2 P_4 arrived at 3 momentum.

Remaining $P_3 \rightarrow$ ~~1~~ done P_3

at time 3 → P_1 P_2 P_4

5 3 2

P_4 যাবে for ~~4~~ momentum, P_5 arrived

Remaing P_4 $P_4 \rightarrow$ ~~2~~ 1

a time 4 → P_1 P_2 ~~P_4~~ P_5

5 3 1 1

এখন P_4 যাবে upto 5 momentum P_6 arrived

Remaining $P_4 \rightarrow$ ~~2~~ ~~1~~

P_6 ~~akki~~

at time 5 → P_1 P_2 P_6 P_5

5 3 3 1

Burst time ছোট P_5 এর so P_5 যাবে

Remaing $P_5 \rightarrow$ ~~1~~ upto 6 momentum

Then P_2 যাবে upto 9 momentum

Remaining $P_2 \rightarrow$ ~~3~~ 4 3

Then P_6 যাবে upto 12 momentum

Remaining $P_6 \rightarrow$ 3

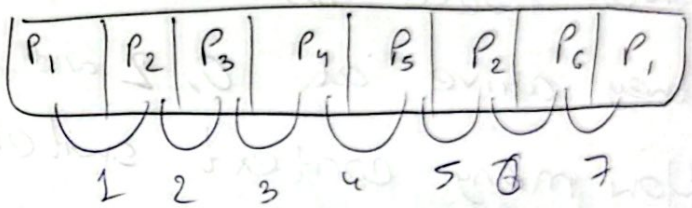
Then P_1 যাবে upto 17 momentum

17 is the completion time

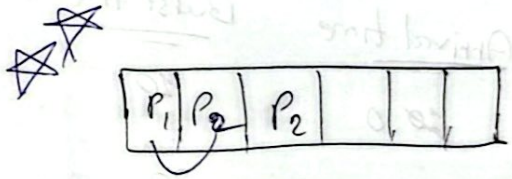
Quest^n: SRTF এ কয়টা context switch হবে?

Ans: Gantt chart এ transition count করব। প্রথম process বাদে

তুমি আরেকটা process ধরলে context switch



So, 7 টা context switch



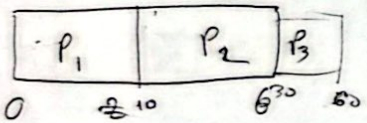
এমন যদি হয় P_2 এর পর P_2 then transi^n count হবে না

MP

Suppose a system is dealing with 3 process which needs 10, 20, 30 time unit and they arrive at 0, 2 and 6. How many context switches needed if the system implements ~~SRST~~ SRTF

Ans:

Process	Arrival time	Burst time
P_1	~~10~~ 0	10
P_2	~~20~~ 2	20
P_3	~~30~~ 6	30



Ready Q	Remaining
0: P_1	$P_1 \rightarrow$ ~~10~~ 8
2: P_1 P_2	$P_2 \rightarrow$ ~~20~~
6: P_1 P_2 P_3	$P_3 \rightarrow$ ~~30~~

Non preemptive | HRRN Highest Response Ratio Next | $RR = \frac{W+S}{S}$

RR → depends if same burst time

Process	Arrival time	Burst time	Completion time	TAT	WT
P_1	0	3	3	3	0
P_2	2	6	9	7	1
P_3	4	4	13	9	5
P_4	6	5	20	14	9
P_5	8	2	15	7	5

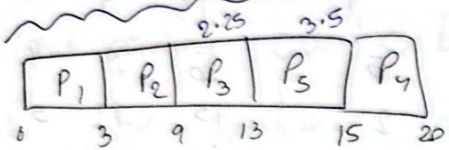
⇒ FCFS এর problem ছিল convo effect so SJF এ shift করবার ফলে 3 issue ছিল burst time less হলে আগে পাবে। burst time যাদের bigger তারা অনেকক্ষণ ধরে starve করছে।

SJF (Non Preemptive) থেকে SRTF (Preemptive) এ shift করবার। Still starvation

↗ আসলে আরও problem আছে।

and prioritization problem persists

Gantt chart



	Ready Queue
at time 0	P_1
" " 3	P_2
" " 9	P_3 P_4 P_5

Avg TAT $= \frac{90}{5}$

Avg WT $= \frac{20}{5}$

at time 9 (3টা process এর calculation)

$RR[P_3] = \frac{[9-4]+4}{4} = 2.25$

$RR[P_4] = \frac{(9-6)+5}{5} = 1.3$

$RR[P_5] = \frac{(9-8)+2}{2} = 1.5$

$RR = \frac{W+S}{S}$ (W → waiting time, S → burst time)

যেটা বড় RR সেটা নিবে

at time 13:

$RR[P_4] = \frac{(13-6)+5}{5} = 2.4$

$RR[P_5] = \frac{(13-8)+2}{2} = 3.5$

$RR = \frac{W+S}{S}$ (W → waiting time, S → burst time)

Response ratio

RR is inversely proportional to burst time

যত বেশি burst time তত RR more which means যত বেশি burst time তত high priority.

HRRN is complementary to SRTF, SJF

SRTF
SJF
এগুলা less burst time কে priority দিত

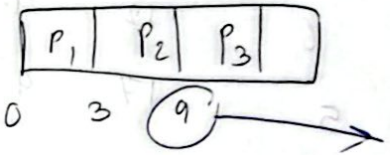
RR is proportional to waiting time (W)
So RR এর through এ waiting time এর সমান priority দিচ্ছে,
So it solved the issue of SRTF, SJF

HRRN depends on RR
এটি RR same এটার depends on burst time
" burst time " " depends on process id

এই moment এ calculate করতে (Gantt chart এ)

$$RR[P_3] = \frac{(9-4) \rightarrow \text{arrival time} + 4}{4}$$

$$= 2.25$$



This is the waiting time of P_3

HRRN solves
1) Starvation

HRRN maintains
1) Fairness
2) Better Response Time

Quest^n 2 scenerio

MP

Find avg TAT and avg WT using SRTF

Process	AT	BT	CT	TAT (CT-AT)	WT (TAT-BT)
P_1	0	12	27	27	15
P_2	2	4	6	4	0
P_3	3	6	12	9	3
P_4	8	5	17	9	4



at time 0 : P_1

" " 2 : P_1, P_2, P_3

" " 3 : P_1, P_2, P_3

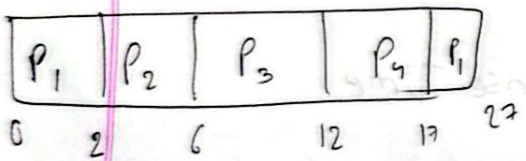
" " 8 : P_1, P_2, P_3, P_4

$P_3 \rightarrow 12 \to 10$

$P_2 \rightarrow 4 \to 2$

$P_3 \rightarrow 6 \to 4$

$P_4 \rightarrow 5$



avg TAT $= \frac{27+4+9+9}{4} = \frac{49}{4}$

avg WT $= \frac{15+0+3+4}{4} = \frac{22}{4}$

Do using HRRN

Process	AT	BT	CT	TAT	WT
P_1	0	12			
P_2	2	4			
P_3	3	6			
P_4	8	5			

Gantt Chart

	P_1	P_2	P_3	P_4
0	12	16	22	27

at time 0 : P_1

" " ~~12~~ : $P_1 P_2$

" " ~~16~~ : $P_1 P_2 P_3$

at time 2 : $P_1 P_2$

" " 3 : $P_1 P_2 P_3$

" " 8 : P_1, P_2, P_3, P_4

Scanned with CamScanner

at time 12

$RR[P_2] = \frac{(12-2)+4}{4} = \frac{14}{4} = 3.5$

$RR[P_3] = \frac{(12-3)+6}{6} = \frac{15}{6} = 2.5$

$RR[P_4] = \frac{(12-8)+5}{5} = \frac{13}{5} = 2.6$

P_2 will go first

~~at time 16~~

$RR[P_3] = \frac{(16-2)+4}{4} = \frac{16}{3} =$

$RR[P_4] = \frac{(16-4)+4}{4}$

At time 16

~~P_3~~

$RR[P_3] = \frac{(16-3)+6}{6} = \frac{19}{6} = 3.167$

$RR[P_4] = \frac{(16-8)+5}{5} = \frac{13}{5} = 2.6$

P_3 will go first

then P_4

Avg TAT -

" WT